



Stream Sentry

RAFAEL PARREIRA CHEQUER

ORIENTADO POR

Cleiton Silva Tavares(TCC 1 - TCC2)

Danilo de Quadros Maia Filho (TCC 1)

Leonardo Vilela (TCC 1)

Raphael Ramos Dias Costa(TCC 1)

Contextualização

Tipo de resolução

- Uma **ferramenta de Engenharia de Software** — um produto de software desenvolvido, e não um método, modelo teórico ou estudo.

Objetivo

- Desenvolver uma ferramenta **web open-source** (React) para automação de **testes end-to-end** em aplicações de **videoconferência** (WebRTC, Whereby, Zoom ou Jitsi).

Concorrentes

<chrome://webrtc-internals>



rtcStats



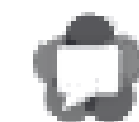
Selenium



Playwright



LOADERO



KITE

Escopo

O Stream Sentry foca em **automação de qualidade** (estabilidade e métricas de QoE) WebRTC com dezenas de usuários virtuais — não em carga em larga escala nem segurança.

Dentro	Fora
Simulação de múltiplos usuários virtuais (Puppeteer, headless)	Testes de estresse massivo (milhares de usuários)
Integração intercambiável: WebRTC, Whereby, Zoom, Jitsi (Strategy)	Plugin nativo de CI/CD (saídas JSON/CSV são consumíveis)
Telemetria WebRTC em tempo real (latência, perdas, bitrate)	Testes de segurança (pentest, interceptação)
Injeção de instabilidade de rede (<i>chaos</i>)	Interface mobile nativa (foco em desktop)
Relatórios exportáveis (JSON/CSV) e histórico auditável	Tecnologias legadas (Flash, RTMP)

Necessidades

Automação de
testes E2E
multi-API

Telemetria de
qualidade em
tempo real

Análise
estruturada e
integração ágil

Funcionalidades

Simulação
múltiplos usuários

Integração
intercambiável

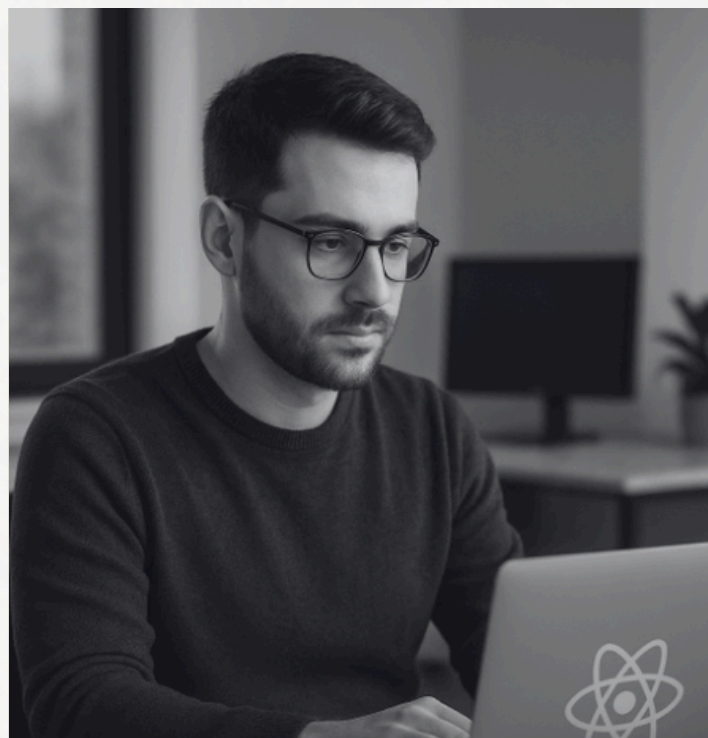
Coleta de
métricas WebRTC

Telemetria ao vivo
por WebSockets

Injeção de
instabilidade

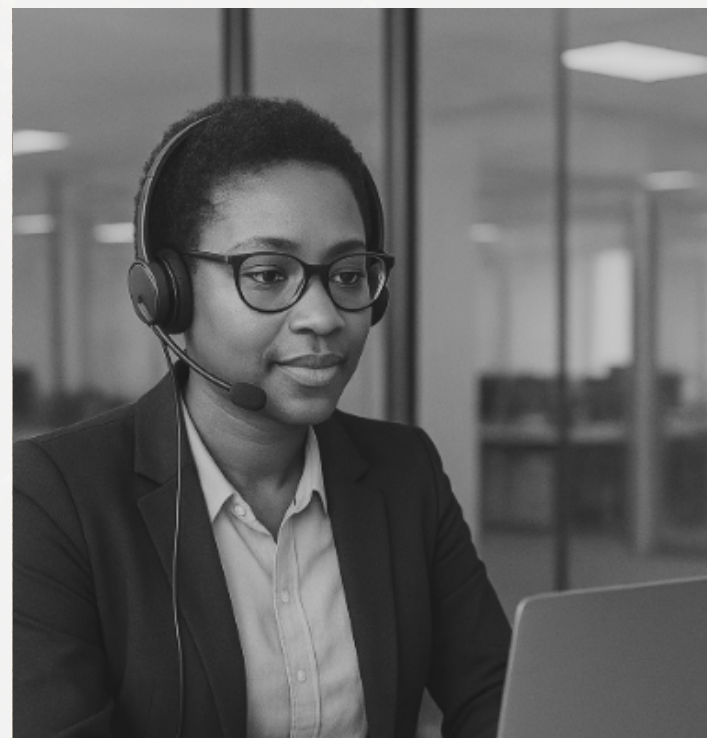
Relatórios
exportáveis

Atores



Lucas

Desenvolvedor React



Ana

Engenheira de QA



Felipe

Tech Lead (Equipe Ágil)

Diagrama de Casos de Uso

Este recorte mostra a autenticação, herdada por todos os atores do Usuário do Sistema, e o Módulo de Simulação. O Desenvolvedor React prepara os testes: **configura os provedores** (Jitsi, Whereby, Zoom ou WebRTC, via padrão Strategy), **simula múltiplos usuários** virtuais e **injeta instabilidade de rede** (perfis de chaos).

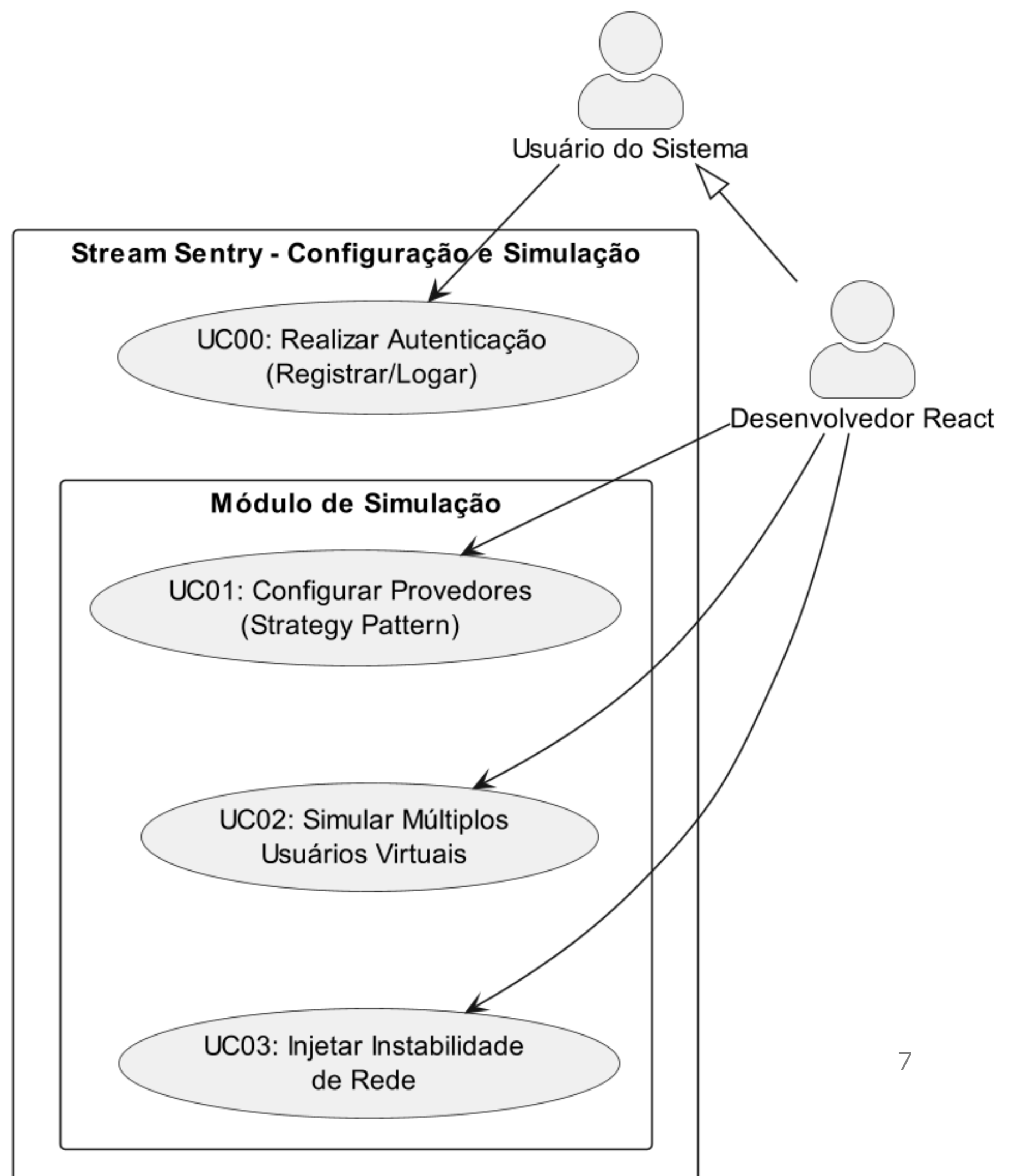
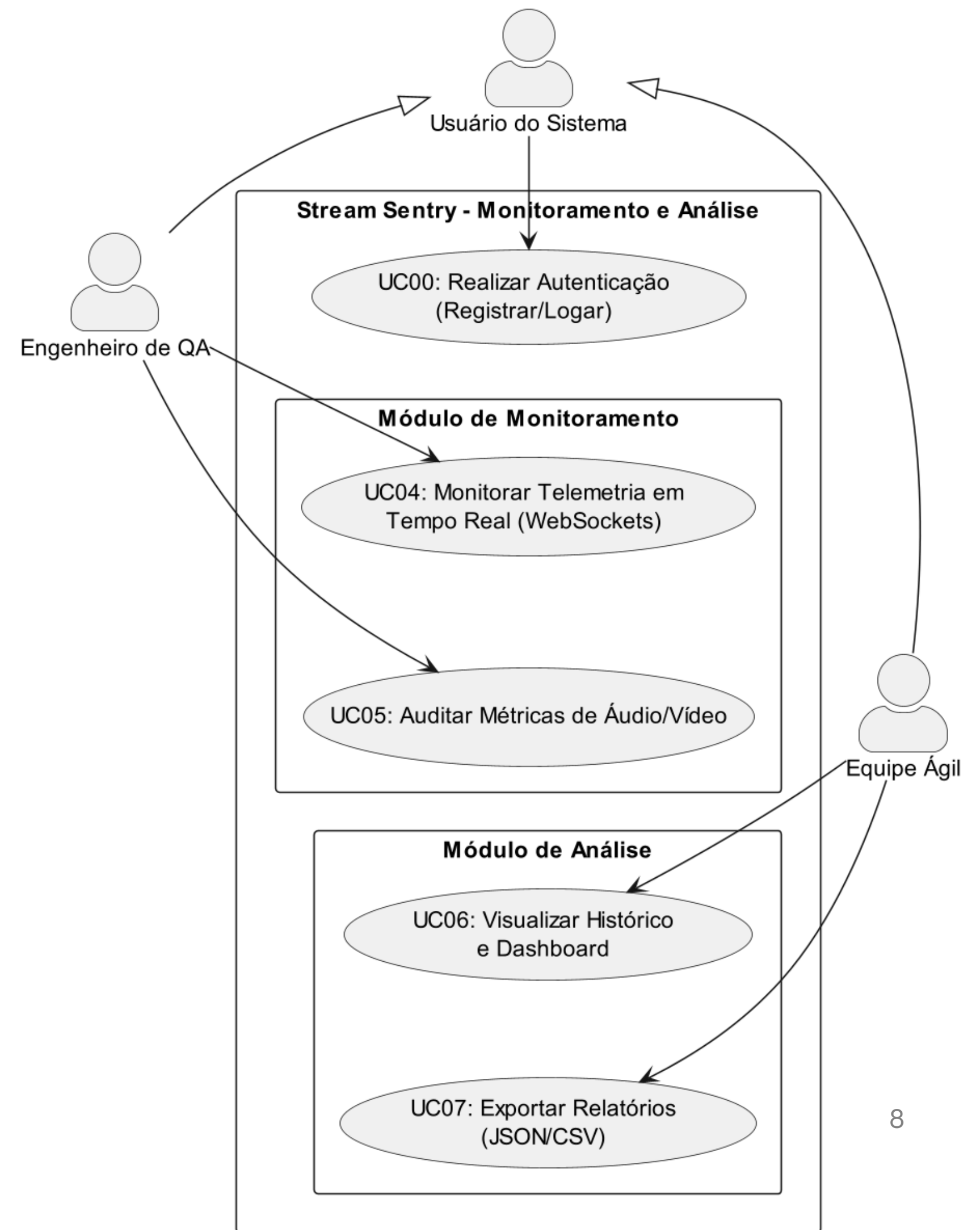


Diagrama de Casos de Uso

Este recorte cobre o que ocorre durante e depois da execução. O Engenheiro de QA atua no Módulo de Monitoramento — **telemetria em tempo real** (WebSockets) e **auditoria de métricas** de áudio/vídeo. A Equipe Ágil atua no Módulo de Análise — **histórico/dashboard** e **exportação de relatórios** (JSON/CSV).

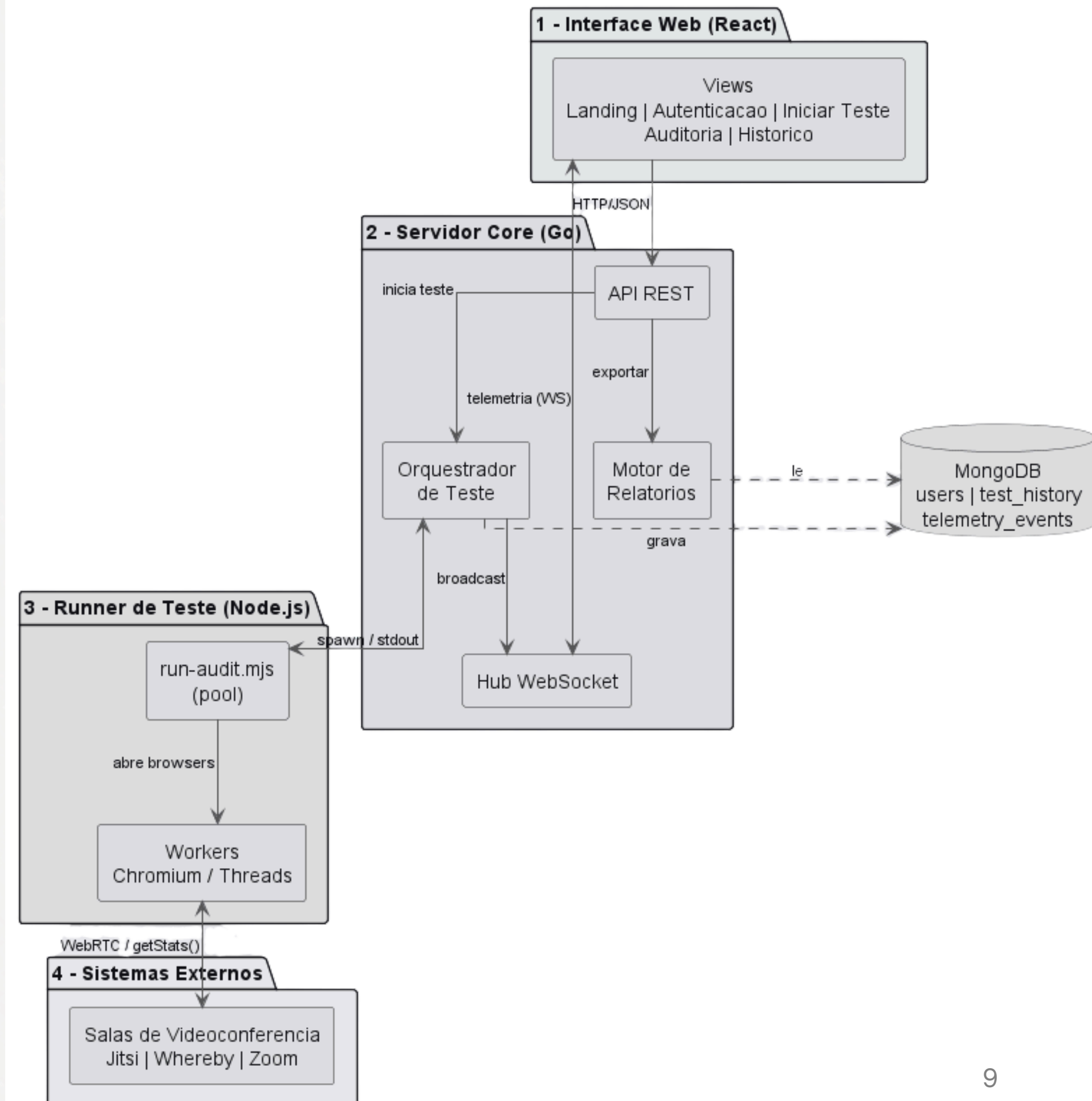
STREAM SENTRY



Modelagem da Arquitetura

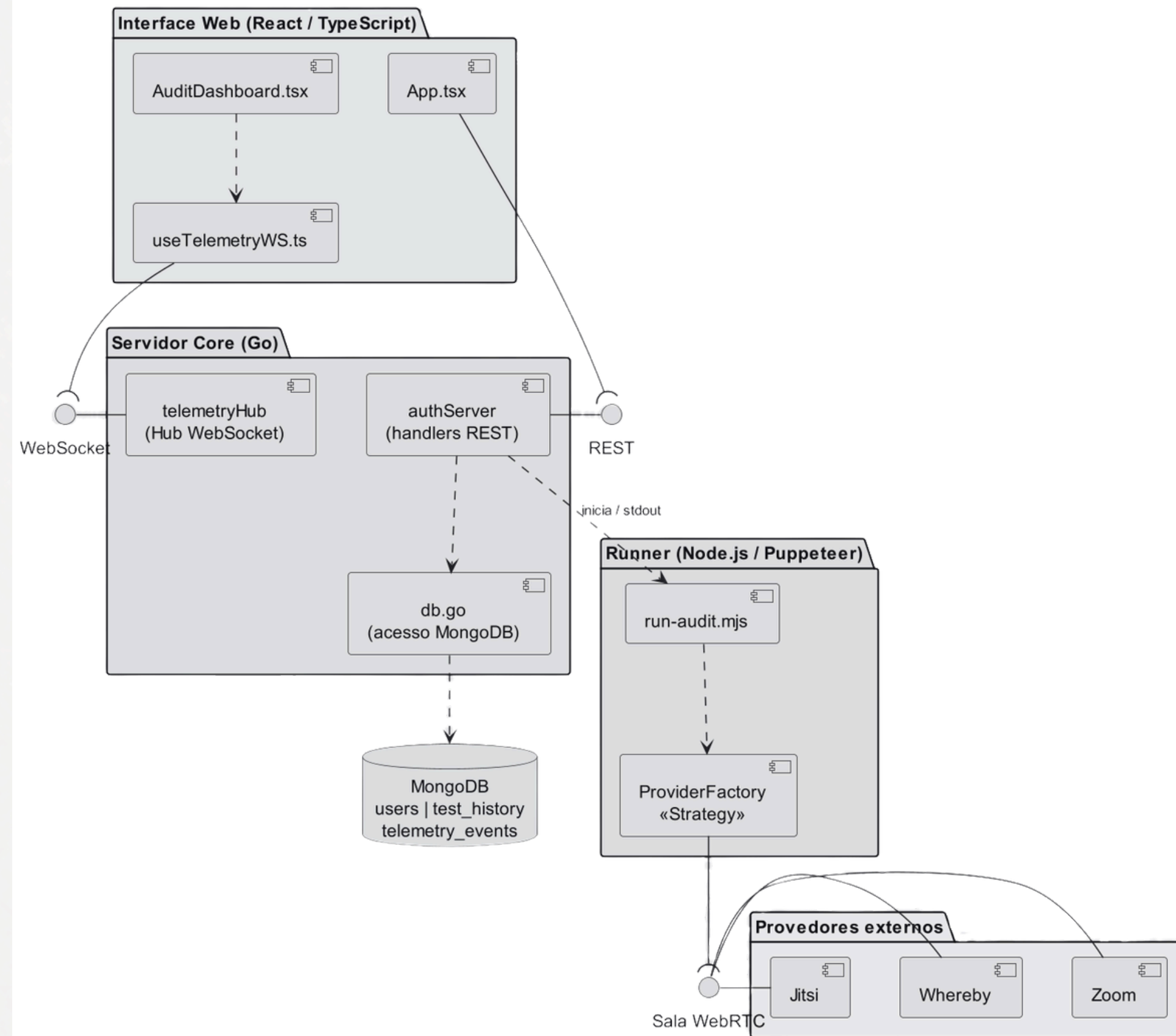
A arquitetura tem **quatro** camadas: **Frontend React (TS)**, **Servidor de Orquestração (Go)** que coordena os testes e agrega as métricas, **Runner Node.js/Puppeteer** que simula os usuários virtuais e coleta a telemetria, e **Persistência (MongoDB)**. A integração com as APIs de vídeo (WebRTC, Whereby, Zoom, Jitsi) usa o padrão Strategy.

STREAM SENTRY



Principais Componentes

A **Interface Web** (React/TS) consome a **API REST do Core** (Go) e recebe a telemetria em tempo real por **WebSocket** (telemetryHub). O Core orchestra o **Runner** (Node.js/Puppeteer), que coleta as **métricas WebRTC** das salas (Jitsi, Whereby, Zoom) com `getStats()` e as devolve pela saída padrão. A **persistência** usa MongoDB (db.go).



Demonstração do Sistema



<https://stream-sentry.com>

Stream Sentry

RAFAEL PARREIRA CHEQUER

Post-Mortem

Provedor	Automação	Métricas	Capacidade	Status
Jitsi	Quase total*	Completas	até 50 VUs*	Escala
Whereby	Total	Completas	4 VUs (grátis)	Pequena escala
Zoom	Inviável (anti-bot)	Parciais	até 10 VUs*	Não recomendado

Acertos

- **Strategy/Factory:** troca de provedor sem mexer no núcleo
- Telemetria via `getStats()` + WebSocket robusta

Lições

- **Desacoplar** é necessidade
- Validar automação antes de fixar o provedor
- Separar limite técnico (Zoom) de comercial (Whereby)

Fim

OBRIGADO PELA ATENÇÃO

Stream Sentry

RAFAEL PARREIRA CHEQUER